coursera | UNIVERSITY OF LONDON

Algorithms and Complexity

Module 1
Automata Theory

Lesson 1.0 Introduction

- Course structure and navigation
  Reading • 15 min
- How to learn effectively on this course
  Reading • 15 min
- Introduction to the course
  Video • 56 sec
- Course Syllabus
  Reading • 10 min

Lesson 1.1 Finite automata

- Introduction
  Video • 1 min
- Basic definitions, letters, strings
  Video • 4 min
- What is an automaton?
  Video • 8 min
- Basics
  Practice Assignment • 30 min

Lesson 1.2 Deterministic automata

- Finite automata – example (part 1)
  Video • 3 min
- Finite automata – example (part 2)
  Video • 8 min
- Working with Automata Simulator
  Video • 6 min
- Design an automata to accept a simple language using Automata Simulator
  Dialogue • 30 min
- Language of the automata
  Video • 4 min
- Recognise a language
  Video • 4 min
- Exercises with hints and tips
  Reading • 1h

Lesson 1.3 Non-deterministic Automata

- Deterministic finite automata (DFA) vs nondeterministic finite automata (NFA)
  Video • 4 min
- DFA example
  Video • 3 min
- DFA example in Automata Simulator
  Video • 4 min
- DFA Simulator
  Lab • 20 min
- Computation by NFA
  Video • 3 min
- NFA example
  Video • 5 min
- NFA example in Automata Simulator
  Video • 2 min
- NFA Simulator
  Lab • 20 min
- Exercises with hints and tips
  Reading • 10 min

Lesson 1.4 Summary and Assessments

- Conclusion
  Video • 3 min
- Check your understanding: End of module 1
  Graded Assignment • Grade: 100%

Module 2
Searching and Sorting Algorithms

Lesson 2.1 Introducing Algorithms

- What is an algorithm ?
  Video • 4 min
- Tell us about an algorithm that you use
  Discussion Prompt • 20 min
- Representing algorithms
  Video • 8 min
- Simple algorithms – insertion sort
  Video • 5 min
- How else would you sort?
  Practice Assignment • 30 min
- Simple algorithms – bubble sort
  Video • 5 min

Lesson 2.2 Sorting and Search Techniques

- Selection sort
  Video • 4 min
- Insertion, bubble and selection sort
  Practice Assignment • 25 min
- When we should use which sorting algorithm?
  Dialogue • 15 min
- Sorting algorithms simulator: Bubble Sort and Insertion Sort
  Lab • 20 min
- Exercises with hints and tips
  Reading • 10 min
- Search techniques
  Video • 4 min
- Binary and sequential search
  Practice Assignment • 12 min

Lesson 2.3 Heap sort

- Binary trees and heaps
  Video • 3 min
- Heapify algorithm
  Video • 4 min
- Heap sort
  Video • 6 min
- Binary trees and heaps
  Reading • 1h
- Heap sort
  Practice Assignment • 20 min

Lesson 2.4: Summary and Assessments

- Conclusion
  Video • 1 min
- Check your understanding: End of module 2
  Graded Assignment • Grade: 100%

Module 3
Recursive and Iterative Algorithms

Lesson 3.1 Recursive and Iterative Algorithms

- Recursion
  Video • 7 min
- Iterative algorithms
  Video • 4 min
- Recursive algorithms example
  Video • 5 min
- Recursive and iterative algorithms
  Practice Assignment • 25 min
- Quick sort
  Video • 6 min
- Quick sort example
  Video • 4 min
- Quick sort
  Practice Assignment • 20 min
- Partition and quick sort algorithms
  Reading • 1h 25m
- Lesson 1 exercises with hints and tips
  Reading • 10 min

Lesson 3.2 Merging

- Merging lists
  Video • 4 min
- How does this magic work?
  Practice Assignment • 30 min
- Merge sort
  Video • 4 min
- Keep merging

Module exercises with hints and tips

0:05 / 2:04 2x

Now it's time to put into practice the concepts that you have learnt so far in this module. Attempt the following exercises. If you get stuck, you can refer to the hints and tips at the bottom of this page.

Please note that these exercises are optional, further practice – but we strongly recommend that you engage with them in order to test your knowledge and see where you might need to do additional study.

1. Assume $f(n) = 3n + 2$ and $g(n) = n^2$, prove if $f(n) = O(g(n))$ or $g(n) = O(f(n))$.

2. What is the time complexity of the following pseudocode?

```
1  i=1
2  while i<=n:
3      print(“Hello”)
4      i=i*2
5
```

3. Assume $f(n) = 4n + logn, g(n) = nlogn$ and $h(n) = n^2$. For each pair of these functions, prove which one belongs to the O of the other, for example, if $f = O(g)$ or $g = O(f)$.

4. What is the time complexity of the following pseudocode?

```
1  i=1
2  while i<=n:
3      print(“Hello”)
4      n=n-i
5      i=i+1
6
```

Hints and tips for each question:

1. f(n)=O(n) and g(n)=O(n2)f(n)=O(n) and g(n)=O(n2)f, left parenthesis, n, right parenthesis, equals, O, left parenthesis, n, right parenthesis, start text, space, a, n, d, space, end text, g, left parenthesis, n, right parenthesis, equals, O, left parenthesis, n, squared, right parenthesis. Now you should be able to decide. Then try to find the witnesses to prove your answer.
2. It is O(logn)O(logn)O, left parenthesis, l, o, g, n, right parenthesis because each time i is doubled.
3. f(n)=O(n)f(n)=O(n)f, left parenthesis, n, right parenthesis, equals, O, left parenthesis, n, right parenthesis, g(n)=O(nlogn)g(n)=O(nlogn)g, left parenthesis, n, right parenthesis, equals, O, left parenthesis, n, l, o, g, n, right parenthesis and h(n)=O(n2)h(n)=O(n2)h, left parenthesis, n, right parenthesis, equals, O, left parenthesis, n, squared, right parenthesis. Now you should be able to decide. Then try to find the witnesses to prove your answer.
4. It is O(n0.5)O(n0.5)O, left parenthesis, n, start superscript, 0, point, 5, end superscript, right parenthesis. Try to trace the algorithm to find the answer.

Mark as completed

Like Dislike Report an issue

Go to next item →